

Homework 4 Written Questions

CSCI 1430

Template Instructions

This document is a template with specific answer regions and a fixed number of pages. Given large class sizes and limited TA time, the template helps the course staff to grade efficiently and still focus on the content of your submissions. Please help us in this task:

- Make this document anonymous.
- Questions are in the orange boxes. Provide answers in the green boxes.
- Use the footer to check for correct page alignment.
- **Do NOT remove the answer box.**
- **Do NOT change the size of the answer box.**
- **Extra pages are not permitted unless otherwise specified.**
- **Template edits or page misalignment will lead to a 10 point deduction.**

Gradescope Submission

- Compile this document to a PDF and submit it to Gradescope.
- Pages will be automatically assigned to the right questions on Gradescope.

This Homework

- 4 questions [**$8 + 18 + 5 + 8 = 39$ points**].
- Include code, images, and equations where appropriate.

Q1 — [8 points]

First, some machine learning warm-up. Given a neural network classifier and the stochastic gradient descent optimizer, discuss how the following variables might affect both the training process and the final outcome (e.g., test accuracy).

If in doubt, declare your assumptions.

Q1.1 — [2 points]

Learning rate. [2–3 sentences]

Q1.2 — [2 points]

Batch size. [2–3 sentences]

Q1.3 — [2 points]

Number of epochs. **[2–3 sentences]**

Q1.4 — [2 points]

Random image augmentations during training, e.g., slight crops, rotations, varying brightness or contrast (color jitter). **[2–3 sentences]**

Q2 — [18 points]

Many traditional computer vision algorithms use convolution to extract feature representations (e.g., SIFT). Convolutional neural networks (CNNs) also use convolution as an inductive bias, giving the vision system specific geometric properties.

In this question, we will explore what these inductive biases provide and what they do not. First, to definitions and assumptions:

- Recall that an operation is *equivariant* to a transformation if transforming the input to that operation produces a correspondingly transformed output. It is *invariant* if the output does not change at all.
- Let's define a *global* property as one that holds across the whole image equally, and a *local* property as one that holds only within a region.
- Assume discrete integer shifts on an integer grid and ignore any boundary effects (e.g. invalid convolution requiring padding for equal size output).

Q2.1 — [3 points]

The Convolutional Layer. Consider a single standard convolutional layer with input size H, W, C , a stride of 1, and with C_{out} filter kernels. Characterize its mathematical relationships with the following input 2D geometry transformations. Place ☐ in all relevant answers.

Transformation	Global Equivariant	Global Invariant	Local Equivariant	Local Invariant	None
Translation by $\Delta x = 1$	☐	☐	☐	☐	☐
Translation by $\Delta x = 2$	☐	☐	☐	☐	☐
Rotation by 30°	☐	☐	☐	☐	☐
Rotation by 90°	☐	☐	☐	☐	☐
Scale by $2.0\times$	☐	☐	☐	☐	☐
Scale by $0.3\times$	☐	☐	☐	☐	☐
Reflection (horizontal flip)	☐	☐	☐	☐	☐

Q2.2 — [3 points]

The Max Pooling Layer. A standard 2×2 max pooling layer with a stride of 2.

Transformation	Global Equivariant	Global Invariant	Local Equivariant	Local Invariant	None
Translation by $\Delta x = 1$	☐	☐	☐	☐	☐
Translation by $\Delta x = 2$	☐	☐	☐	☐	☐
Rotation by 30°	☐	☐	☐	☐	☐
Rotation by 90°	☐	☐	☐	☐	☐
Scale by $2.0\times$	☐	☐	☐	☐	☐
Scale by $0.3\times$	☐	☐	☐	☐	☐
Reflection (horizontal flip)	☐	☐	☐	☐	☐

Q2.3 — [3 points]

The Average Pooling Layer. A standard 2×2 average pooling layer with a stride of 2.

Transformation	Global Equivariant	Global Invariant	Local Equivariant	Local Invariant	None
Translation by $\Delta x = 1$	☐	☐	☐	☐	☐
Translation by $\Delta x = 2$	☐	☐	☐	☐	☐
Rotation by 30°	☐	☐	☐	☐	☐
Rotation by 90°	☐	☐	☐	☐	☐
Scale by $2.0\times$	☐	☐	☐	☐	☐
Scale by $0.3\times$	☐	☐	☐	☐	☐
Reflection (horizontal flip)	☐	☐	☐	☐	☐

Q2.4 — [3 points]

The Global Average Pooling Layer. A layer that takes the mean of all spatial dimensions: $(H, W, C) \rightarrow (C)$, resulting in a 1D vector of length C . Such layers are often used at the end of encoder blocks to provide fixed-length generic features, e.g., to present a $C = 256$ -dimensional feature to a task-specific head regardless of the prior encoder architecture. For example, `torch.nn.AdaptiveAvgPool2d(1)`.

Transformation	Global Equivariant	Global Invariant	Local Equivariant	Local Invariant	None
Translation by $\Delta x = 1$	☐	☐	☐	☐	☐
Translation by $\Delta x = 2$	☐	☐	☐	☐	☐
Rotation by 30°	☐	☐	☐	☐	☐
Rotation by 90°	☐	☐	☐	☐	☐
Scale by $2.0\times$	☐	☐	☐	☐	☐
Scale by $0.3\times$	☐	☐	☐	☐	☐
Reflection (horizontal flip)	☐	☐	☐	☐	☐

Q2.5 — [3 points]

Putting it all together. In a real-world computer vision task, a CNN can succeed with objects at different translations, rotations, scales, and reflections. With respect to the learned filter kernels, how does a CNN accommodate these? **[4–6 sentences]**

Translation:
Rotation:
Scale:
Reflection:

Q2.6 — [3 points]

In a CNN, when is translation equivariance maintained through average pooling? When through max pooling? What effect might this have on classifying a translating input? **[4–6 sentences]**

Another way to think about it: Suppose I classify birds. If a bird translates within the image but is otherwise identical, will I still receive the same confidence of bird class?

Note: It might help to relate different pooling functions to downsampling, and to consider what we know about downsampling, image frequency, and aliasing.

Average:
Max:

Q3 — [5 points]

Let's return to max pooling. What effects are caused by adding a spatial max pooling layer (stride ≥ 2) between two convolutional layers, where the output with max pooling is some size larger than $1 \times 1 \times C$?

There may be multiple correct answers per question.

Q3.1 — [1 point]

What happens to the computational cost of training? (Ignore the cost of the max pooling layer itself.)

- ☐ Increases
- ☐ Stays the same
- ☐ Decreases

Q3.2 — [1 point]

What happens to the computational cost of testing? (Ignore the cost of the max pooling layer itself.)

- ☐ Increases
- ☐ Stays the same
- ☐ Decreases

Q3.3 — [1 point]

In a classification task, what happens to the complexity of the decision boundary?

Note: here we do not mean computational complexity.

- ☐ Increases
- ☐ Stays the same
- ☐ Decreases

Q3.4 — [1 point]

What happens to the potential for overfitting?

- ☐ Increases
- ☐ Stays the same
- ☐ Decreases

Q3.5 — [1 point]

What happens to the potential for underfitting?

- ☐ Increases
- ☐ Stays the same
- ☐ Decreases

Q4 — [8 points]

Neural networks are sensitive to the scale and distribution of intermediate activations. *Normalization layers* attempt to stabilize training by controlling these distributions. There are many approaches, each with different effects.

Consider two approaches, applied after a convolutional layer with C output channels, spatial size $H \times W$, and a mini-batch of B images:

Approach A — Instance Normalization (Ulyanov et al., 2016). For each image independently, normalize each channel to zero mean and unit variance:

$$\hat{x}_{n,c} = \frac{x_{n,c} - \mu_{n,c}}{\sqrt{\sigma_{n,c}^2 + \epsilon}},$$

where $\mu_{n,c}$ and $\sigma_{n,c}^2$ are the mean and variance computed over the spatial dimensions ($H \times W$) of image n , channel c only. ϵ is a small constant to stop div. by zero.

Approach B — Batch Normalization (Ioffe & Szegedy, 2015). For each channel, normalize using statistics computed across the entire mini-batch:

$$\hat{x}_{n,c} = \frac{x_{n,c} - \mu_c}{\sqrt{\sigma_c^2 + \epsilon}}, \quad y_{n,c} = \gamma_c \hat{x}_{n,c} + \beta_c$$

where μ_c and σ_c^2 are computed over all B images and all spatial positions ($H \times W$) for channel c . Then, define learnable parameters γ_c and β_c that produce an affine transform to rescale and reshift the batch distribution per channel.

Q4.1 — [3 points]

Suppose we train two CNNs to classify photographs as either daytime or nighttime, one each using approach A and approach B for normalization. Upon validating our classifier, we find that the network with approach A failed but the B network succeeded.

Why did the A network fail, and why did the B network succeed? Under what conditions within our data loader would the B network also fail? **[3–4 sentences]**

Q4.2 — [2 points]

At test time, we often process one image at a time ($B = 1$). Approach A has no problem here, but Approach B requires batch statistics. Explain how batch normalization handles single-image inference. **[2–3 sentences]**

Q4.3 — [3 points]

In the code portion, you will train on a small dataset with high data variance. Choosing to use normalization layers (and which ones) can have a significant effect on our result.

As we know, batch normalization estimates population statistics from each mini-batch. When the underlying data has high variance, the per-batch statistics μ_c and σ_c^2 can fluctuate significantly from batch to batch.

Would you expect this to affect the *first* convolutional layer (which sees raw pixels) more or less than *deeper* layers? Why? **[3–5 sentences]**

Feedback? (Optional)

We appreciate your feedback on how to improve the course. You can provide anonymous feedback through [this form](#) which can be accessed using your Brown account (your identity will not be collected). If you have urgent non-anonymous comments/questions, please email the instructor.